# TP 12 : Gestion des Fichiers en Android

Développement Mobile – BTS

## Objectifs pédagogiques du TP

- Compréhension du stockage interne
- Manipulation des flux Java
- Parsing CSV
- Gestion des exceptions
- Architecture séparée (UI / logique)

## I. Partie Théorique

### 1. Le stockage dans Android

Android propose plusieurs types de stockage :

- **Stockage interne** : privé à l'application
- **Stockage externe** : partagé
- **Base de données SQLite**
- **Stockage cloud (Firebase)**

Dans ce TP, nous utilisons le **stockage interne**.
Caractéristiques :

- Accessible uniquement par l'application
- Aucune permission nécessaire
- Persistant après fermeture de l'application
- Supprimé lors de la désinstallation

Le fichier est stocké dans :

```
/data/data/nom.package/files/
```

### 2. Manipulation des fichiers en Java

Pour manipuler un fichier, nous utilisons des flux (Streams).

### Écriture

```
FileOutputStream fos =
context.openFileOutput("notes.txt", Context.MODE_APPEND);
```

Modes possibles :

- MODE_PRIVATE : écrase le fichier
- MODE_APPEND : ajoute à la fin

Écriture optimisée :

```
BufferedWriter writer =
new BufferedWriter(new OutputStreamWriter(fos));
```

### Lecture

```
FileInputStream fis =
context.openFileInput("notes.txt");

BufferedReader reader =
new BufferedReader(new InputStreamReader(fis));
```

Lecture ligne par ligne :

```
while ((line = reader.readLine()) != null) {
    String[] parts = line.split(";");
}
```

## 3. Format des données

Les données sont stockées en format CSV :

id ; title ; description ; language

Pourquoi utiliser un séparateur ?

- Permet de reconstruire un objet
- Facilite le parsing
- Simple et léger

## 4. Gestion des Exceptions

Deux exceptions principales :

- FileNotFoundException
- IOException

Structure :

```
try {
   // code
} catch (IOException e) {
   e.printStackTrace();
}
```

Pourquoi gérer les exceptions ?

- Éviter le crash
- Gérer fichier inexistant
- Gérer erreurs de lecture/écriture

## 5. UUID

UUID permet de générer un identifiant unique :

```
String id = UUID.randomUUID().toString();
```

Pourquoi ne pas utiliser le titre comme clé ?

- Deux titres peuvent être identiques
- Le titre peut être modifié
- L'identifiant doit rester stable

## 6. GridView

GridView permet d'afficher une liste dynamique sous forme de grille (lignes et colonnes).

Composants :

- Adapter (BaseAdapter)
- Méthode getView()
- numColumns (nombre de colonnes)
- notifyDataSetChanged()

Chaque élément affiché est généré dans la méthode `getView()`, qui associe les données à la vue correspondante.

GridView devient scrollable automatiquement si le nombre d'éléments dépasse la taille de l'écran.

```
<GridView
    android:id="@+id/gridView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:numColumns="1"
    android:verticalSpacing="10dp"
    android:horizontalSpacing="10dp"
    android:stretchMode="columnWidth"/>
```

- **android :id** : identifiant permettant d'accéder au GridView dans le code Java.
- **layout_width="match_parent"** : occupe toute la largeur de l'écran.
- **layout_height="match_parent"** : occupe toute la hauteur disponible.
- **numColumns="1"** : nombre de colonnes affichées (1 colonne = affichage vertical).
- **verticalSpacing** : espace entre les lignes.
- **horizontalSpacing** : espace entre les colonnes.
- **stretchMode="columnWidth"** : ajuste la largeur des colonnes pour remplir l'espace disponible.

# II. Partie Pratique

## Objectif

Développer une application permettant :

- Ajouter une note
- Afficher toutes les notes
- Supprimer une note
- Mettre à jour une note

## Structure des classes

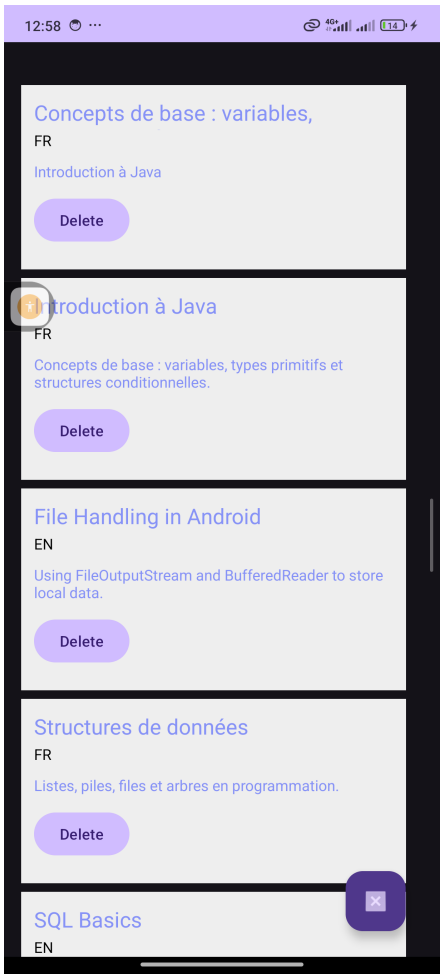
- DataClass.java : modèle
- FileManager.java : logique fichier
- MainActivity.java : affichage liste
- AddActivity.java : ajout
- NoteAdapter.java : gestion RecyclerView

## Travail demandé

1. Implémenter la méthode saveNote()
2. Implémenter readNotes()
3. Tester la persistance après redémarrage
4. Implémenter deleteNote()
5. Implémenter updateNote()
6. Ajouter validation des champs vides

## Descriptions des fichiers et méthodes

- **DataClass.java**
  Cette classe représente le modèle de données d'une note et permet de structurer les informations stockées dans le fichier.
- **FileManager.java**
  Cette classe contient toute la logique de gestion des fichiers (lecture, écriture, suppression et mise à jour), séparée de l'interface utilisateur.
- **saveNote()**
  Cette méthode permet d'ajouter une nouvelle note dans le fichier texte en mode ajout (*append*).
- **readNotes()**
  Cette méthode lit le fichier ligne par ligne et reconstruit les objets `DataClass` à partir du format CSV.
- **deleteNote()**
  Cette méthode supprime une note en réécrivant entièrement le fichier sans la ligne correspondante.




(a) Liste RecycleView contenant les données

(b) Ajout d'une nouvelle note

FIGURE 1 – Interfaces de l'application

— **updateNote()**
Cette méthode met à jour une note en réécrivant complètement le fichier avec les nouvelles informations.

— **MainActivity.java**
Cette activité affiche la liste des notes à l'aide d'un `GridView` et recharge les données à chaque retour à l'écran.

— **NoteAdapter.java**
Cet adaptateur permet de lier les données (`DataClass`) à l'interface graphique du `GridView`.

— **activity_add.xml**
Ce layout définit le formulaire permettant à l'utilisateur d'ajouter une nouvelle note.

— **activity_main.xml**
Ce layout contient le `GridView` affichant les notes ainsi qu'un bouton flottant pour ajouter une nouvelle note.

— **item_note.xml**
Ce layout définit l'apparence d'une seule note dans la liste affichée par le `GridView`.

# III. Code : Implémentation Guidée

## Objectif

Développer une application Android permettant :

- Ajouter une note
- Lire toutes les notes
- Supprimer une note
- Mettre à jour une note

Les données seront stockées dans un fichier interne : **notes.txt**
Format des données :

id ; title ; description ; language

## 1. DataClass.java (Fourni)

Ce fichier est déjà fourni.

```java
public class DataClass {

    private String key;
    private String dataTitle;
    private String dataDesc;
    private String dataLang;

    public DataClass(String key,
                     String dataTitle,
                     String dataDesc,
                     String dataLang) {
        this.key = key;
        this.dataTitle = dataTitle;
        this.dataDesc = dataDesc;
        this.dataLang = dataLang;
    }

    public String getKey() { return key; }
    public String getDataTitle() { return dataTitle; }
    public String getDataDesc() { return dataDesc; }
    public String getDataLang() { return dataLang; }
}
```

## 2. MainActivity.java

Cette activité affiche toutes les notes enregistrées à l'aide d'un `GridView`.

### A. Déclaration des composants

```java
public class MainActivity extends AppCompatActivity {

    GridView gridView;
    NoteAdapter adapter;
    FloatingActionButton btnAdd;
}
```

**Explication :**

- `GridView` : affiche les notes sous forme de grille.
- `NoteAdapter` : relie les données à l'interface.
- `FloatingActionButton` : permet d'ajouter une nouvelle note.

### B. Méthode onCreate()

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    gridView = findViewById(R.id.gridView);
    btnAdd = findViewById(R.id.btnAdd);

    loadData();

    btnAdd.setOnClickListener(v ->
        startActivity(new Intent(this, AddActivity.class)));
}
```

**Explication :**

- `setContentView()` : charge l'interface graphique.
- `findViewById()` : relie les composants XML au code Java.
- `loadData()` : charge les notes depuis le fichier.
- Le bouton ouvre `AddActivity` pour ajouter une note.

### C. Méthode loadData()

```java
private void loadData() {

    // ======================================
    // À COMPLETER :
    // Lire les données depuis le fichier
    // en utilisant FileManager.readNotes(...)
    List<DataClass> list = ...
    // ======================================

    // ======================================
    // À COMPLETER :
    // Initialiser  l adapter
    adapter = new ..........
```

```
    // ========================================

    // ========================================
    // À COMPLETER :
    // Associer l adapter au GridView
    // ========================================
}
```

**Pourquoi créer une méthode loadData() ?**

- Éviter de répéter le même code plusieurs fois.
- Centraliser le chargement des données.
- Faciliter la mise à jour de l'affichage.

### D. Gestion du Long Click (Modifier)

Dans la méthode loadData() :

```
gridView.setOnItemLongClickListener((parent, view, position, id) ->
    {

    // ========================================
    // À COMPLETER :
    // Récupérer l objet correspondant
    // à la position sélectionnée
    DataClass note = (DataClass) adapter.getItem(position);
    // ========================================

    // ========================================
    // À COMPLETER :
    // Ouvrir UpdateActivity et transmettre
    // les données via Intent
    // ========================================

    return true; // important pour indiquer que l'événement est
        consommé
});
```

**Objectif :**
Permettre à l'utilisateur de modifier une note en maintenant le doigt appuyé.
Le **return True** signifie que l'événement long click a été traité, sinon Android pourrait déclencher un autre événement.

### E. Pourquoi utiliser onResume() ?

```
@Override
protected void onResume() {
    super.onResume();
    loadData();
}
```

**Explication :**

- `onResume()` est appelée lorsque l'activité redevient visible.

— Après l'ajout ou la modification d'une note, on revient à `MainActivity`.
— Il est donc nécessaire de recharger les données.

**Pourquoi ne pas utiliser uniquement notifyDataSetChanged() ?**

— `notifyDataSetChanged()` actualise l'affichage si les données en mémoire ont changé.
— Dans notre cas, les données sont relues depuis un fichier.
— Il est donc plus simple de recréer l'adapter via `loadData()`.

## 3. FileManager.java

### 1. Implémentation de saveNote()

Compléter la méthode suivante dans FileManager.java :

```
public static void saveNote(Context context,
                            String title,
                            String desc,
                            String lang) {

    try {

        FileOutputStream fos =
           -------------------------------------;

        BufferedWriter writer =
            -------------------------------------;

        String id =
            UUID.--------------------------------------;

        // ==================================================
        // À COMPLETER :
        // Écrire la ligne au format :
        // id ; title ; desc ; lang
        // Ajouter un saut de ligne
        // ==================================================


        writer.close();

    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

**Questions :**

— Pourquoi utilise-t-on MODE_APPEND ?
— Pourquoi générer un UUID ?

### 2. Implémentation de readNotes()

Compléter la méthode suivante :

```
public static List<DataClass> readNotes(Context context) {

    List<DataClass> list = new ArrayList<>();

    try {

        FileInputStream fis =
            --------------------------------------;

        BufferedReader reader =
            --------------------------------------;

        String line;

        while ((--------------------------------------) {

            String[] parts = line.split(";");

            if (parts.length == 4) {

                // ==========================================
                // À COMPLETER :
                // Reconstruire un objet DataClass
                // Ajouter l'objet à la liste
                // ==========================================

            }
        }

        reader.close();

    } catch (------------------------) {
        // Fichier inexistant
    } catch (------------------------) {
        e.printStackTrace();
    }

    ------------------------;
}
```

**Question :**
Pourquoi tester `parts.length == 4` ?

### 3. Implémentation de deleteNote()

Rappel :
Un fichier texte ne permet pas de supprimer une ligne directement.
Algorithme :

1. Lire toutes les notes

2. Ouvrir le fichier en MODE_PRIVATE
3. Réécrire toutes les notes sauf celle à supprimer

Compléter :

```
public static void deleteNote(Context context,
                              String id) {

    ------------------------ notes =
        readNotes(context);

    try {

        FileOutputStream fos =
            context.openFileOutput(FILE_NAME,
               ------------------------);

        BufferedWriter writer =
            ------------------------;

        for (DataClass n : notes) {

            // ========================================
            // À COMPLETER :
            // Si l'id est différent
            // Réécrire la ligne
            // ========================================

        }

        writer.close();

    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

**Question :**
Pourquoi utilise-t-on MODE_PRIVATE ici ?

### 4. Implémentation de updateNote()

Algorithme :

1. Lire toutes les notes
2. Si l'id correspond → modifier
3. Sinon → réécrire normalement

Compléter :

```
public static void updateNote(Context context,
                              String id,
                              String newTitle,
                              String newDesc,
```

```
                                String newLang) {

    List<DataClass> notes =
        readNotes(context);

    try {

        FileOutputStream fos =
            ------------------------;

        BufferedWriter writer =
           ------------------------;

        for (------------:------------) {

            // ======================================
            // À COMPLETER :
            // Si id correspond     écrire nouvelle valeur
            // Sinon     écrire ancienne valeur
            // ======================================

            writer.newLine();
        }

        writer.close();

    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

**Question :**
Pourquoi la mise à jour nécessite la réécriture complète du fichier ?

## 4. Validation des champs

Dans AddActivity, empêcher l'enregistrement si un champ est vide.
À compléter :

```
btnSave.setOnClickListener(v -> {

    String title = uploadTopic.getText().toString();
    String desc  = uploadDesc.getText().toString();
    String lang  = uploadLang.getText().toString();

    // ==================================
    // À COMPLETER :
    // Vérifier champs vides
    // Afficher Toast si nécessaire
    // ==================================
    FileManager.saveNote(
    this,
```

```
    ------------------------,
    ------------------------,
    ------------------------,
    );
    Toast.------------------------;
    finish();

});
```

**Rôle de `finish()`** :

La méthode `finish()` permet de fermer l'Activity actuelle et de revenir à l'Activity précédente. Dans ce cas, après l'enregistrement d'une note, l'écran du formulaire (`AddActivity`) se ferme automatiquement et l'utilisateur revient à la liste principale. Cela permet d'actualiser l'affichage et d'améliorer l'expérience utilisateur.

## 5. Implémentation de UpdateActivity

L'activité `UpdateActivity` permet de modifier une note existante.
Elle utilise le même layout que `AddActivity` mais avec un comportement différent.

## Étape 1 : Charger les données reçues

Compléter le code suivant :

```java
public class UpdateActivity extends AppCompatActivity {

    EditText uploadTopic, uploadDesc, uploadLang;
    Button btnSave;

    String key;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_add);

        uploadTopic = findViewById(R.id.uploadTopic);
        uploadDesc  = findViewById(R.id.uploadDesc);
        uploadLang  = findViewById(R.id.uploadLang);
        btnSave     = findViewById(R.id.saveButton);

        // ======================================
        // À COMPLETER :
        // Récupérer les données envoyées
        // par Intent (key, title, desc, lang)
        // ======================================

        // ======================================
        // À COMPLETER :
        // Pré-remplir les champs EditText
```

```
        // ======================================

        btnSave.setText("Update");
    }
}
```

**Consigne :**
Utiliser `getIntent().getStringExtra(...)` pour récupérer les données.

### Étape 2 : Modifier la note

Compléter l'action du bouton :

```
btnSave.setOnClickListener(v -> {

    String newTitle = uploadTopic.getText().toString();
    String newDesc  = uploadDesc.getText().toString();
    String newLang  = uploadLang.getText().toString();

    // ======================================
    // À COMPLETER :
    // Appeler FileManager.updateNote(...)
    // avec les nouvelles valeurs
    // ======================================

    // ======================================
    // À COMPLETER :
    // Fermer l'activité avec finish()
    // ======================================
});
```

### Question de réflexion

Pourquoi est-il nécessaire de transmettre la `key` et non seulement le titre ?

## Travail Final

1. Tester l'ajout de 5 notes
2. Fermer et relancer l'application
3. Vérifier la persistance des données
4. Tester suppression et mise à jour

## Questions de réflexion

1. Quelle différence entre MODE_APPEND et MODE_PRIVATE ?
2. Pourquoi un fichier texte ne permet pas la suppression directe ?
3. Quand utiliser un fichier au lieu d'une base SQLite ?
4. Quels sont les avantages et limites du stockage interne ?

# Annexes : Ressources XML

## 1 1. Drawable : lavender_border.xml

À placer dans : `res/drawable/`

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">

    <stroke
        android:width="2dp"
        android:color="@color/lavender"/>

    <corners
        android:radius="30dp"/>

</shape>
```

Ce fichier définit un arrière-plan personnalisé avec une bordure violette et des coins arrondis, utilisé pour styliser les champs de saisie.

## 2 2. colors.xml

À placer dans : `res/values/`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>

    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>

    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>

    <color name="lavender">#8692f7</color>
    <color name="red">#c70000</color>
    <color name="green">#00ff00</color>

</resources>
```

Ce fichier centralise toutes les couleurs utilisées dans l'application afin de faciliter la maintenance et la cohérence visuelle.

## 3 3. themes.xml

À placer dans : `res/values/`

```xml
<resources xmlns:tools="http://schemas.android.com/tools">

    <style name="Base.Theme.TP12_FILES"
        parent="Theme.Material3.DayNight.NoActionBar">

        <item name="colorPrimary">@color/lavender</item>
        <item name="colorPrimaryVariant">@color/lavender</item>
        <item name="colorOnPrimary">@color/white</item>

        <item name="colorSecondary">@color/teal_200</item>
        <item name="colorSecondaryVariant">@color/teal_700</item>
        <item name="colorOnSecondary">@color/black</item>

        <item name="android:statusBarColor">
            ?attr/colorPrimaryVariant
        </item>

    </style>

    <style name="Theme.TP12_FILES"
        parent="Base.Theme.TP12_FILES" />

    <style name="roundedImageViewRounded">
        <item name="cornerFamily">rounded</item>
        <item name="cornerSize">50%</item>
    </style>

</resources>
```

Ce fichier définit le thème global de l'application (couleurs principales, style général et apparence de la barre de statut).

# Conclusion

Ce TP permet de comprendre :

- Le fonctionnement réel des fichiers
- La manipulation des flux
- La gestion des erreurs
- La séparation logique/UI
- Les bases de l'architecture Android